

**UNIVERSIDAD AUTÓNOMA DEL BENI
“JOSÉ BALLIVIAN”**

**FACULTAD DE INGENIERÍA Y TECNOLOGÍA
INGENIERÍA DE SISTEMAS**



APLICACIÓN WEB CON PAAS, GIT/GITHUB Y DESPLIEGUE WEB

Estudiantes:

- Cristhian Anderson Roca Saucedo
- Meloddy Ortiz Roca
- Yerko Alex Suárez Banegas

Docente: Ing. Hermes Rodriguez Rivero

Asignatura: Tecnologías Emergentes

Semestre: 7º Semestre

Fecha: 06/2026

Trinidad - Beni – Bolivia

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo General.....	3
2.2. Objetivos Específicos.....	3
3. Herramientas Utilizadas.....	4
4. Descripción de la Arquitectura Cloud Implementada	4
4.1 Flujo General y Conectividad del Sistema.....	4
4.2 Guía de Implementación Paso a Paso	5
5. Clasificación del Modelo de Servicio (IaaS/PaaS/SaaS).....	8
5.1 Infrastructure as a Service (IaaS).....	9
5.2 Supabase: Platform as a Service (PaaS) / Backend as a Service (BaaS)	9
5.3 Vercel: Platform as a Service (PaaS) enfocado en Frontend	10
5.4 GitHub: Software as a Service (SaaS)	10
6. Ventajas y Limitaciones Encontradas.....	10
6.1 Ventajas de la Arquitectura Implementada.....	10
6.2 Limitaciones de la Arquitectura Encontradas	11
7. Reflexión Final - Respuestas a las Preguntas Guía.....	12
8. Conclusiones y Reflexión Grupal	14
8.1 Conclusiones	14
8.2 Reflexión Grupal y Aprendizaje Colaborativo	15

1. Introducción

El desarrollo web moderno ha experimentado una transformación radical debido al auge de *Cloud Computing* (Computación en la Nube). Tradicionalmente, desplegar una aplicación requería configurar servidores físicos, sistemas operativos, bases de datos locales y complejas redes de comunicación. Hoy en día, los modelos de servicio en la nube permiten a los ingenieros abstraerse de la infraestructura física para enfocarse exclusivamente en el código y en la lógica de negocio.

En este informe se detalla la construcción e implementación de una aplicación web. Esta aplicación web demuestra de forma práctica cómo interactúan múltiples servicios en la nube (Backend, Bases de Datos, Sincronización y Hosting) sin haber tenido que configurar ningún servidor físico local.

2. Objetivos

2.1. Objetivo General

Desarrollar una aplicación web funcional que integre un backend en tiempo real utilizando Supabase como servicio PaaS, gestionar el código fuente con Git y GitHub, y realizar su despliegue en producción mediante Vercel; aplicando los conceptos de Cloud Computing vistos en clase.

2.2. Objetivos Específicos

- Comprender el rol de PaaS en el ciclo de vida del desarrollo de software.
- Crear y configurar un proyecto en Supabase como backend gestionado en la nube.
- Implementar operaciones CRUD y tiempo real con Supabase Realtime.
- Aplicar control de versiones con Git y colaboración en GitHub.
- Desplegar y poner en producción la aplicación usando Vercel.
- Reflexionar sobre las ventajas del modelo PaaS respecto al despliegue tradicional.

3. Herramientas Utilizadas

Para el correcto desarrollo e implementación del Sistema de Registro de Tareas, se integró un ecosistema de herramientas de software orientadas a la filosofía DevOps y el desarrollo ágil en la nube, divididas de la siguiente manera:

Herramienta	Función	URL / Instalación
Visual Studio Code	Editor de código fuente	https://code.visualstudio.com (gratuito)
Node.js 20 LTS	Entorno de ejecución JavaScript	https://nodejs.org (gratuito)
Git	Control de versiones	https://git-scm.com (gratuito)
GitHub	Repositorio remoto y colaboración	https://github.com (cuenta gratuita)
Supabase	BaaS/PaaS: DB + Auth + Realtime	https://supabase.com (tier gratuito)
Vercel	Despliegue y hosting en la nube	https://vercel.com (hobby tier gratuito)

4. Descripción de la Arquitectura Cloud Implementada

4.1 Flujo General y Conectividad del Sistema

La arquitectura del Sistema de Registro de Tareas Colaborativas se basa en un modelo descentralizado y completamente alojado en la nube. En lugar de depender de un servidor físico local, el sistema interconecta tres capas principales de manera síncrona:

1. Capa de Control de Versiones y Repositorio (Git / GitHub): Centraliza el código fuente del proyecto y actúa como el disparador automático para la fase de despliegue continuo.
2. Capa de Backend y Datos (Supabase): Provee una base de datos relacional PostgreSQL y expone servicios en tiempo real a través de WebSockets, que permite la actualización inmediata de la interfaz de usuario sin recargas manuales de la página.
3. Capa de Hosting Frontend (Vercel): Distribuye la interfaz gráfica (HTML, CSS y JavaScript) a nivel global mediante una red de entrega de contenidos (CDN) y ejecuta flujos automatizados de publicación.

4.2 Guía de Implementación Paso a Paso

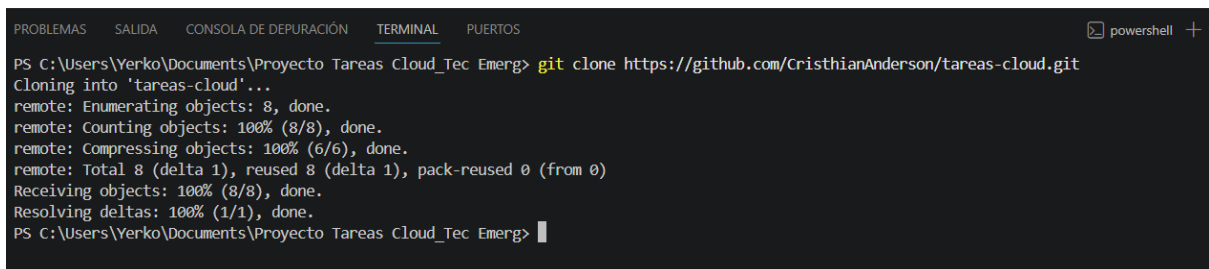
A continuación, se detalla el proceso técnico configurado por el equipo para la puesta en marcha de la infraestructura cloud, incluyendo las evidencias del proceso:

Paso 1: Configuración del Repositorio Local y Remoto (Git & GitHub)

Se inicializa un repositorio Git local en la máquina de desarrollo para registrar el historial de cambios del código fuente. Posteriormente, el Administrador del equipo crea un repositorio público en GitHub para centralizar el flujo de trabajo colaborativo y permitir el acceso a los roles de Desarrollador y Documentador para la descarga del entorno.

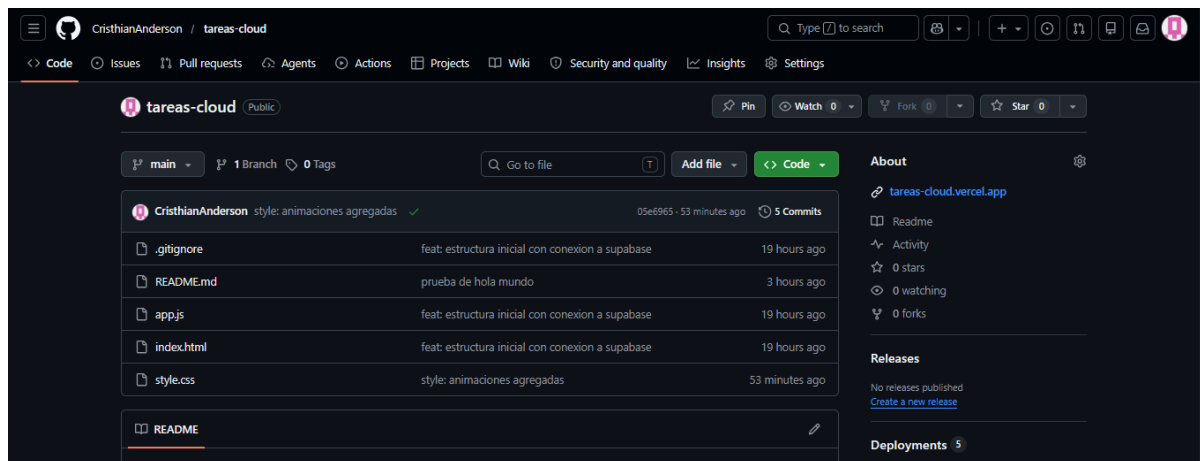
Comandos utilizados en la terminal:

```
git init
git add .
git commit -m "Initial commit - Base del Proyecto"
git remote add origin https://github.com/CristhianAnderson/tareas-cloud.git
git push -u origin main
```

A screenshot of a PowerShell terminal window. The title bar shows 'PROBLEMAS', 'SALIDA', 'CONSOLA DE DEPURACIÓN', 'TERMINAL', and 'PUERTOS'. The terminal content shows the command 'git clone https://github.com/CristhianAnderson/tareas-cloud.git' being executed. The output indicates the repository is being cloned into 'tareas-cloud', with progress for enumerating, counting, and compressing objects, and finally receiving the objects and resolving deltas. The prompt at the end is 'PS C:\Users\Yerko\Documents\Proyecto Tareas Cloud_Tec Emerg>'.

```
PS C:\Users\Yerko\Documents\Proyecto Tareas Cloud_Tec Emerg> git clone https://github.com/CristhianAnderson/tareas-cloud.git
Cloning into 'tareas-cloud'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 1), reused 8 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.
Resolving deltas: 100% (1/1), done.
PS C:\Users\Yerko\Documents\Proyecto Tareas Cloud_Tec Emerg>
```

Clonado exitoso usando git clone



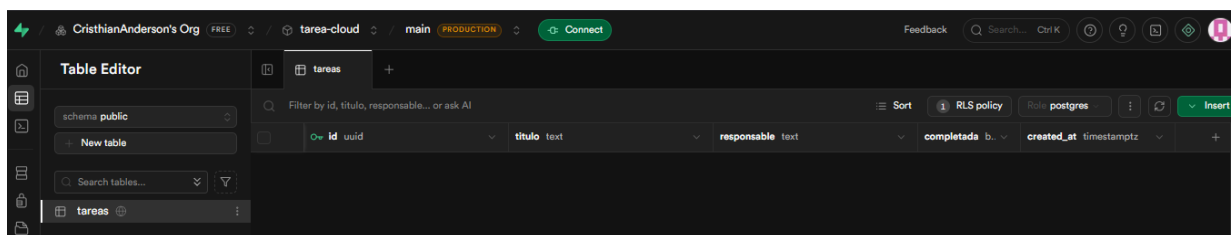
Repositorio Principal en GitHub

Paso 2: Creación del Backend y Base de Datos (Supabase)

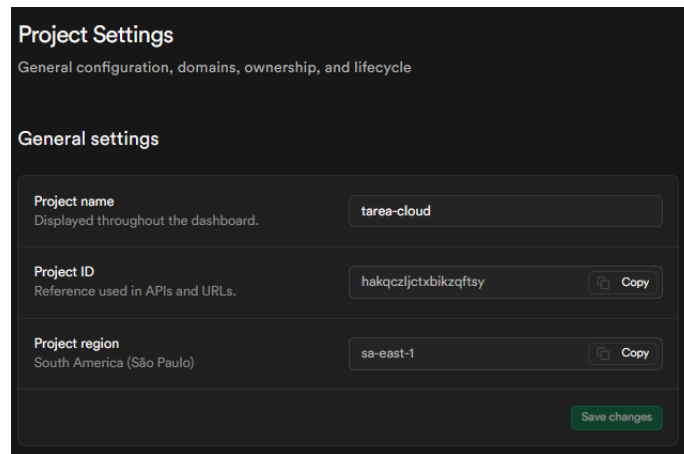
Se crea un proyecto en la plataforma Supabase. Dentro del panel de administración, se diseña la tabla de base de datos relacional (PostgreSQL) encargada de almacenar las tareas del equipo. Además, se habilita de manera explícita la opción de Realtime (Tiempo Real) para permitir que los clientes escuchen los eventos de inserción y eliminación instantáneamente.

Estructura de la Tabla (tasks):

- id: columna que sirve como el identificador único de cada tarea. Tipo de dato *Universally Unique Identifier*.
- título: Nombre del campo donde se almacena la descripción o nombre de la actividad a realizar (Text).
- responsable: campo destinado a guardar el nombre de la persona o integrante del grupo asignado para cumplir la tarea (Text).
- completada: Verificar si la tarea está completada o pendiente (boolean).
- created_at: Campo encargado del registro cronológico (timestamp).

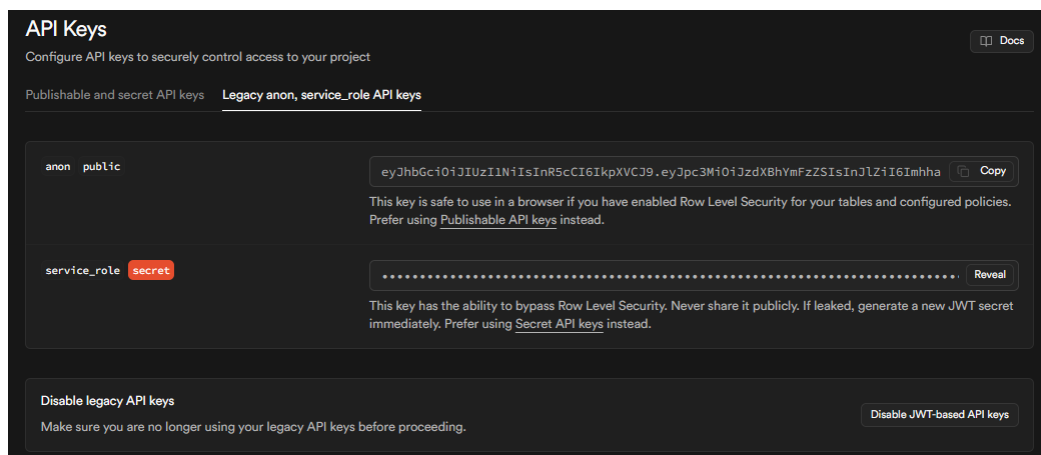


Panel de control de Supabase - Table Editor



The screenshot shows the 'Project Settings' page in Google Cloud Platform. The subtitle is 'General configuration, domains, ownership, and lifecycle'. Under the 'General settings' section, there are three fields: 'Project name' with the value 'tarea-cloud', 'Project ID' with the value 'hakqczljctxbikzqftsy' and a 'Copy' button, and 'Project region' with the value 'sa-east-1' and a 'Copy' button. A 'Save changes' button is at the bottom right.

Project ID: hakqczljctxbikzqftsy

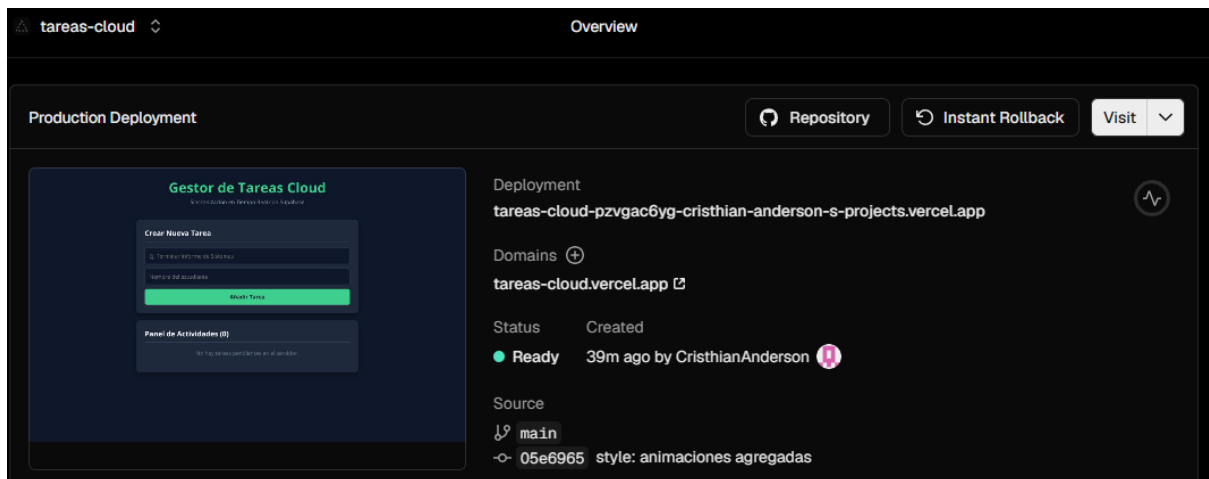


The screenshot shows the 'API Keys' page in Google Cloud Platform. The subtitle is 'Configure API keys to securely control access to your project'. There are two tabs: 'Publishable and secret API keys' and 'Legacy anon, service_role API keys'. The 'Legacy anon, service_role API keys' tab is active. It shows two keys: 'anon public' with a long alphanumeric string and a 'Copy' button, and 'service_role secret' with a masked string and a 'Reveal' button. Below the keys, there are two buttons: 'Disable legacy API keys' and 'Disable JWT-based API keys'.

Anon Public Key

Paso 3: Despliegue Automatizado e Integración Continua (Vercel)

Se vincula la cuenta de GitHub con la plataforma de hosting Vercel. Se importa el repositorio del proyecto seleccionando la rama main. Vercel configura de forma automática el flujo de despliegue continuo: cada vez que un integrante del equipo sube un cambio a GitHub, Vercel lo detecta, procesa la aplicación y actualiza la página web en producción en cuestión de segundos, asignando un dominio seguro con protocolo HTTPS.



Repositorio importado en Vercel



Aplicación Web Desplegada en el Navegador

5. Clasificación del Modelo de Servicio (IaaS/PaaS/SaaS)

Para comprender el impacto y la eficiencia de la infraestructura seleccionada, es fundamental analizar los componentes del sistema según la clasificación estándar de los modelos de servicio

en Cloud Computing. Esta división determina el nivel de responsabilidad, gestión y abstracción que el equipo de desarrollo delega en los proveedores de la nube.

5.1 Infrastructure as a Service (IaaS)

La Infraestructura como Servicio (IaaS) es un modelo de computación en la nube en el que un proveedor externo facilita recursos de hardware virtualizados a través de Internet, tales como servidores físicos, espacio de almacenamiento en disco, conexiones de red y componentes de enrutamiento. Bajo este esquema, el usuario se desentiende del mantenimiento del hardware, pero asume la responsabilidad absoluta de instalar, configurar y actualizar el sistema operativo, los firewalls de seguridad y los motores de bases de datos necesarios para la aplicación.

En el desarrollo de este proyecto, no se utilizó el modelo IaaS de forma directa. Toda la arquitectura web implementada se estructuró sobre soluciones de abstracción superior, específicamente Plataforma como Servicio (PaaS) mediante Supabase y Vercel. Esto evitó que el equipo tuviese que aprovisionar máquinas virtuales o instalar de forma manual el motor de base de datos PostgreSQL en un servidor dedicado.

No obstante, existe una relación indirecta, ya que la infraestructura de hardware que sostiene los servicios PaaS contratados se ejecuta de fondo sobre los centros de datos e infraestructura física (IaaS) de proveedores globales como Amazon Web Services (AWS).

5.2 Supabase: Platform as a Service (PaaS) / Backend as a Service (BaaS)

Supabase se clasifica categóricamente como Plataforma como Servicio (PaaS), y de manera más específica dentro de la industria, como Backend como Servicio (BaaS).

El proveedor se encarga por completo de la gestión de la infraestructura física, el aprovisionamiento de servidores, la instalación del sistema operativo, el mantenimiento del motor de la base de datos (PostgreSQL) y la configuración de las redes de comunicación. El equipo de desarrollo no tiene que preocuparse por parches de seguridad del servidor ni por la escalabilidad del hardware; en su lugar, interactúa directamente con una plataforma lista para crear tablas, gestionar la autenticación y consumir APIs automáticas a través de código JavaScript.

5.3 Vercel: Platform as a Service (PaaS) enfocado en Frontend

Vercel opera bajo el modelo de Plataforma como Servicio (PaaS), estando altamente optimizado para el despliegue de aplicaciones web modernas y entornos frontend.

Vercel abstrae por completo la complejidad de configurar servidores web (como Nginx o Apache), la gestión de certificados de seguridad SSL (HTTPS) y la distribución global de archivos a través de redes CDN. El equipo simplemente conecta su repositorio de código y la plataforma se encarga de compilar, optimizar y poner en línea la interfaz de usuario de forma automatizada. El control del sistema operativo subyacente queda en manos del proveedor, permitiendo a los desarrolladores centrarse exclusivamente en la lógica del cliente y el diseño visual.

5.4 GitHub: Software as a Service (SaaS)

GitHub se clasifica como Software como Servicio (SaaS), representando la capa superior de abstracción en nuestro ecosistema cloud.

Es una aplicación web completa y terminada que el equipo utiliza directamente a través del navegador o de herramientas cliente (como Git) sin necesidad de programar, compilar o desplegar su código base. GitHub proporciona una interfaz lista para el usuario final que resuelve el almacenamiento de repositorios, el control de accesos, la gestión de ramas y las herramientas de colaboración visual. Toda la infraestructura técnica, el backend de la plataforma y el mantenimiento de la aplicación son responsabilidad absoluta de Microsoft/GitHub.

6. Ventajas y Limitaciones Encontradas

El análisis del impacto operativo al implementar una arquitectura basada en modelos de servicio en la nube permite identificar tanto los beneficios estratégicos obtenidos por el equipo como los desafíos técnicos implícitos en este tipo de entornos tecnológicos.

6.1 Ventajas de la Arquitectura Implementada

- **Velocidad de despliegue y desarrollo:** La abstracción de la infraestructura permitió al equipo configurar una base de datos relacional síncrona y un servidor de alojamiento

web en cuestión de poco tiempo. Esto eliminó la necesidad de instalar, configurar y asegurar software de servidor localmente.

- **Sincronización colaborativa en tiempo real nativa:** Gracias a la arquitectura BaaS de Supabase, la integración de WebSockets se realizó de forma transparente a través de su librería. El sistema actualiza la lista de tareas de todos los usuarios conectados de manera inmediata sin necesidad de programar sockets complejos desde cero.
- **Automatización de procesos mediante CI/CD:** La vinculación entre GitHub y Vercel eliminó las tareas manuales de subida de archivos (como el uso tradicional de protocolos FTP). Cada confirmación de cambio en el código fuente genera una compilación automática que minimiza los errores humanos en el despliegue.
- **Ecosistema gratuito y accesible:** El uso de los niveles gratuitos (free tiers) de herramientas profesionales como GitHub, Supabase y Vercel facilitó el desarrollo integral de una aplicación con estándares de la industria sin incurrir en costos económicos para el equipo.

6.2 Limitaciones de la Arquitectura Encontradas

- **Dependencia absoluta de la conectividad a internet:** Al estar toda la lógica de datos y el hosting centralizados en la nube, el entorno de desarrollo y la aplicación en producción quedan completamente inoperables ante fallas en el servicio de internet local.
- **Restricciones de los planes gratuitos (Free Tiers):** Las plataformas utilizadas imponen límites estrictos para proyectos sin costo. Supabase limita el espacio de almacenamiento, el número de conexiones simultáneas y pausa los proyectos tras periodos de inactividad, mientras que Vercel restringe el ancho de banda mensual y los minutos de compilación.
- **Pérdida de control sobre la infraestructura subyacente:** El equipo no tiene acceso ni capacidad de configuración sobre el sistema operativo, las variables del servidor web o la optimización profunda del motor de base de datos. Cualquier caída global en los centros de datos de Vercel o Supabase interrumpe el servicio sin que el equipo pueda intervenir.
- **Curva de aprendizaje técnica en Git:** El flujo de trabajo colaborativo requiere una disciplina estricta en el uso de comandos de consola. Errores menores en el manejo de

ramas o confirmaciones de cambio pueden generar conflictos en el código fuente que detienen el pipeline de despliegue automatizado.

7. Reflexión Final - Respuestas a las Preguntas Guía

1. ¿En qué modelo de servicio (IaaS/PaaS/SaaS) clasificarías a Supabase y por qué?

Supabase se clasifica bajo el modelo de Plataforma como Servicio (PaaS), y específicamente como Backend as a Service (BaaS). Esto se debe a que el proveedor abstrae por completo la infraestructura de hardware, redes y la administración del sistema operativo subyacente. El equipo de desarrollo no realiza tareas de mantenimiento del servidor ni de instalación del motor de base de datos; en su lugar, recibe una plataforma lista con herramientas integradas (base de datos PostgreSQL, autenticación y APIs) consumibles de manera directa mediante código.

2. ¿Qué tareas del backend manejó Supabase que normalmente tendrías que hacer manualmente?

En un entorno de desarrollo tradicional, el equipo habría tenido que programar de forma manual las siguientes tareas que Supabase gestionó automáticamente:

- Aprovisionamiento, instalación y configuración del motor de base de datos relacional PostgreSQL en un servidor dedicado.
- Desarrollo completo de una API REST utilizando lenguajes de servidor (como Node.js, Python o PHP) para crear las rutas de comunicación que permitan leer e insertar los registros de tareas.
- Configuración de controladores y listeners para habilitar la transmisión de datos bidireccional y síncrona.
- Implementación de sistemas de seguridad y filtrado de consultas directamente en el servidor web.

3. ¿Cómo funciona el tiempo real en Supabase? ¿Qué tecnología subyacente utiliza?

El tiempo real en Supabase funciona mediante una arquitectura orientada a eventos que utiliza WebSockets como tecnología de comunicación subyacente. Supabase incorpora un servicio en su infraestructura que escucha el registro de transacciones (Write-Ahead Log o WAL) de la base de datos PostgreSQL. En el momento exacto en que un usuario inserta o modifica una

tarea, el sistema captura el cambio en la base de datos y lo "empuja" (push) instantáneamente a través de un canal bidireccional de WebSocket que permanece abierto con todos los navegadores de los usuarios conectados, actualizando la interfaz de usuario en milisegundos sin requerir una recarga de la página.

4. ¿Cuál fue el proceso de CI/CD que implementaste con GitHub y Vercel?

El proceso de Integración Continua y Despliegue Continuo (CI/CD) se estructuró a través del siguiente pipeline automatizado:

- **Integración Continua (CI):** Los cambios locales en el código fuente realizados por el equipo se registraron mediante comandos Git y se fusionaron (push) en el repositorio centralizado en GitHub, para asegurar que todo el equipo trabaje sobre una base de código única, unificada y libre de conflictos.
- **Despliegue Continuo (CD):** Vercel se configuró para monitorear en tiempo real la rama principal (main) de GitHub. Al detectar un nuevo cambio subido, Vercel disparó automáticamente un flujo que compiló los archivos HTML, CSS y JavaScript, los optimizó y los distribuyó en sus servidores de producción globales, actualizando la aplicación web de cara al usuario en cuestión de segundos y sin interrupciones del servicio.

5. ¿Qué ventajas y desventajas encontraste en el modelo PaaS comparado con un servidor tradicional?

- **Ventajas de PaaS:** Proporciona una velocidad de desarrollo y puesta en producción (despliegues en muy poco tiempo), elimina por completo la carga operativa de administrar hardware, sistemas operativos o parches de seguridad del servidor, cuenta con escalabilidad automática ante picos de tráfico y ofrece una integración nativa de flujos automatizados de despliegue (CI/CD).
- **Desventajas de PaaS:** Dependencia absoluta de la conexión a internet para el desarrollo y el consumo del servicio, pérdida total de control y personalización sobre las variables internas del servidor o del motor de base de datos, y restricciones rígidas o costos elevados en las cuotas de almacenamiento y ancho de banda al superar los límites de los planes gratuitos.

6. ¿Qué consideración de seguridad deberías aplicar si esta fuera una aplicación en producción real?

Para migrar este proyecto académico a un entorno empresarial o comercial legítimo, se deberían implementar con carácter obligatorio las siguientes medidas de seguridad:

- Activación y configuración estricta de las políticas de Seguridad a Nivel de Filas (RLS - Row Level Security) en Supabase, garantizando que un usuario autenticado solo pueda ver, editar o borrar sus propias tareas y no las de otros miembros.
- Implementación de un sistema robusto de Autenticación de Usuarios (JWT / OAuth), restringiendo el acceso público general a las credenciales y obligando el inicio de sesión seguro.
- Protección de las llaves de la API utilizando variables de entorno ocultas del lado del servidor o restricciones de origen HTTP (CORS) en el panel de Supabase, para bloquear peticiones maliciosas que provengan de dominios web no autorizados.
- Encriptación avanzada de datos sensibles en tránsito y en reposo dentro de la base de datos PostgreSQL.

8. Conclusiones y Reflexión Grupal

Tras la finalización del laboratorio y la puesta en producción del Sistema de Registro de Tareas Colaborativas, el equipo de trabajo ha llegado a las siguientes conclusiones:

8.1 Conclusiones

- **Eficiencia operativa de los modelos PaaS:** La adopción de tecnologías basadas en Plataforma como Servicio (PaaS), a través de Supabase y Vercel, demostró ser una estrategia óptima para el desarrollo ágil de software. La capacidad de delegar la administración de la infraestructura y el aprovisionamiento de bases de datos permitió al equipo reducir los tiempos de configuración inicial y enfocar los mayores esfuerzos en la lógica de negocio y la interfaz de usuario.
- **Viabilidad de las arquitecturas Serverless y Realtime:** La implementación de WebSockets mediante Supabase validó que las aplicaciones modernas demandan una sincronización de datos síncrona y transparente. El flujo continuo de datos sin recargas de página no solo eleva la experiencia del usuario, sino que demuestra la madurez de

los servicios BaaS (Backend as a Service) para manejar eventos concurrentes en tiempo real.

- **Impacto de la filosofía DevOps y CI/CD:** La integración de Git, GitHub y Vercel sirvió como un modelo práctico de automatización del ciclo de vida del software. Establecer un pipeline automatizado de despliegue continuo eliminó la fricción operativa y los errores humanos asociados con las subidas manuales de archivos, garantizando que el entorno de producción refleje de forma inmediata y segura los cambios aprobados por el equipo.

8.2 Reflexión Grupal y Aprendizaje Colaborativo

El desarrollo de esta práctica representó una experiencia que nos mostró el contraste de los conceptos teóricos de Cloud Computing con las exigencias del entorno profesional real. Pasar de conceptos teóricos a la implementación y despliegue de una aplicación web funcional y accesible en internet, nos brindó una perspectiva mucho más amplia sobre las tendencias actuales en tecnología.

Durante este proceso, comprendimos que el uso de Git y GitHub trasciende el simple almacenamiento o respaldo de archivos; por el contrario, requiere una disciplina estricta en el flujo de trabajo local y remoto. Gestionar los comandos desde la terminal y coordinar las confirmaciones de cambio nos enseñó la importancia de la sincronización para evitar conflictos en el código fuente que pudieran comprometer el avance del equipo.

Uno de los aspectos más destacables de la práctica fue constatar la eficiencia operativa que ofrecen los modelos de servicio actuales. Analizar la evolución tecnológica y notar cómo las plataformas PaaS (Supabase y Vercel) permiten estructurar una base de datos relacional y desplegar un entorno de producción global en cuestión de minutos, abstrayendo por completo la compleja configuración de servidores locales tradicionales, evidencia el impacto de la nube en la optimización del tiempo. Asimismo, presenciar la sincronización inmediata de las tareas entre los diferentes navegadores del equipo mediante WebSockets consolidó nuestro entendimiento sobre la viabilidad de las arquitecturas en tiempo real.

Finalmente, este proyecto nos deja la satisfacción de haber gestionado un ciclo de despliegue completo de forma autónoma. Nos llevamos como aprendizaje principal que el desarrollo moderno no se limita únicamente a la escritura de código, sino a la capacidad estratégica de integrar soluciones cloud para trabajar de manera ágil, colaborativa e inteligente.